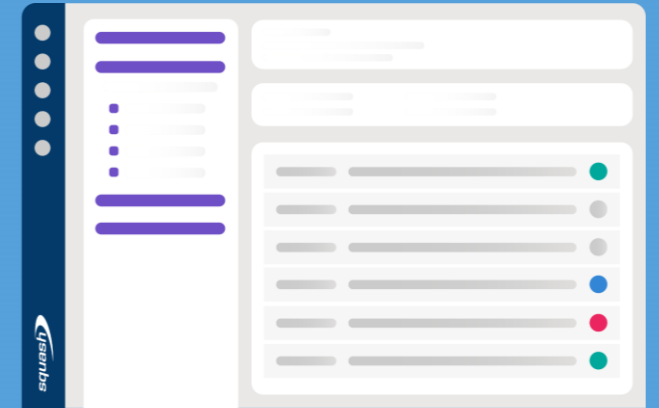


Les fondamentaux du test fonctionnel



3. Techniques de conception des tests



Synonymes :

- techniques de conception de cas de test;
- techniques de spécification de test;
- techniques de conception de test.

(Glossaire ISTQB)

- Lors de la conception des cas de test, de nombreux paramètres entrent en ligne de compte et peuvent **démultiplier le volume de cas de test** à formaliser (tests multi-navigateurs, habilitations et profils, matériels etc.).

Par exemple, un SUT nécessitant d'exécuter 150 cas de test à réaliser selon le profil (Admin/utilisateur), à tester sur Edge + Firefox + Chrome + Safari. Théoriquement, on exécuterait $150 \times 2 \times 4 = 1200$ CT ...

- Les techniques de conception de tests ont pour but de **déterminer l'approche la plus productive** pour **réduire l'effort de test*** tout en maintenant un **niveau de couverture maximal**.

** Charge, exprimée en jH*

- L'**objectif** d'une technique de conception de tests est d'aider à **identifier** :
 - **les conditions** de test,
 - **les cas** de test et
 - **les données** de test.

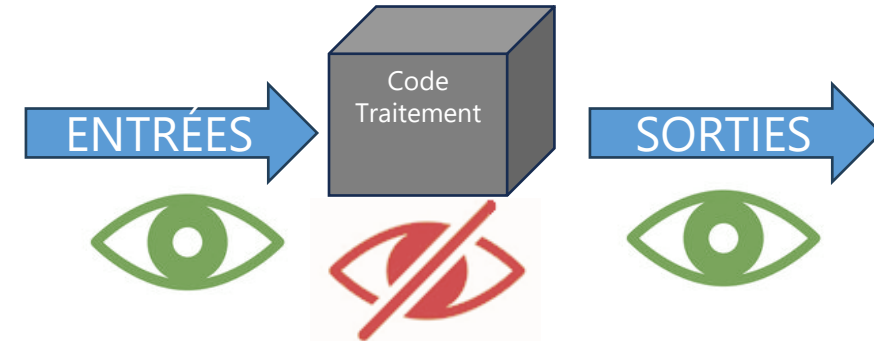
- Le choix des techniques de test à utiliser dépend d'un **certain nombre de facteurs**, dont les suivants :
 - Type de **composant** ou de système
 - **Complexité** du composant ou des systèmes
 - **Normes** réglementaires
 - **Exigences** client ou contractuelles
 - Niveaux de **risque**
 - Types de risques
 - **Objectifs** du test
 - Connaissances et compétences des **testeurs**
 - **Outils** disponibles
 - **Temps** et **budget**
 - **Modèle** de cycle de vie du développement logiciel
 - **Utilisation** prévue du logiciel
 - **Expérience** antérieure de l'utilisation des techniques de test sur le composant ou le système à tester
 - Types de **défauts** attendus dans le composant ou le système
 - **Documentation** disponible

Les 3 catégories de techniques de test :

- **Techniques de boîte-noire** : Via l'analyse de la base de test (spécifications, User Stories, Exigences...).

Elles se concentrent sur les entrées et sorties de l'objet de test sans référence à sa structure interne.

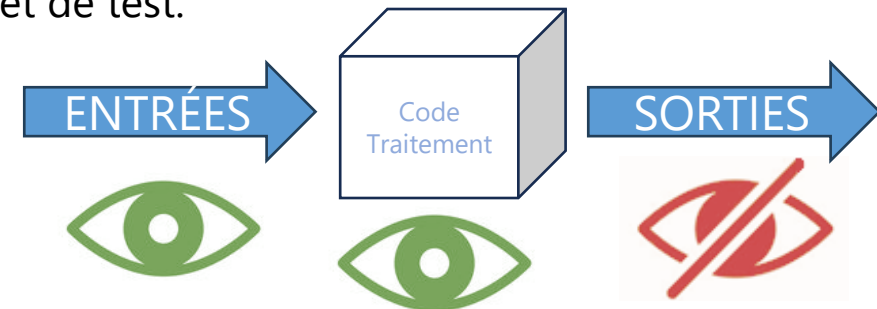
Ex: *Partitions d'équivalence, analyse des valeurs limites, tables de décisions, transitions d'état, cas d'utilisation*



- **Techniques de boîte-blanche** : Via l'analyse de l'architecture, de la structure interne, du code.

Elles se concentrent sur la structure et le traitement à l'intérieur de l'objet de test.

Ex: *Couverture des décisions, couverture des instructions, instructions et décisions*



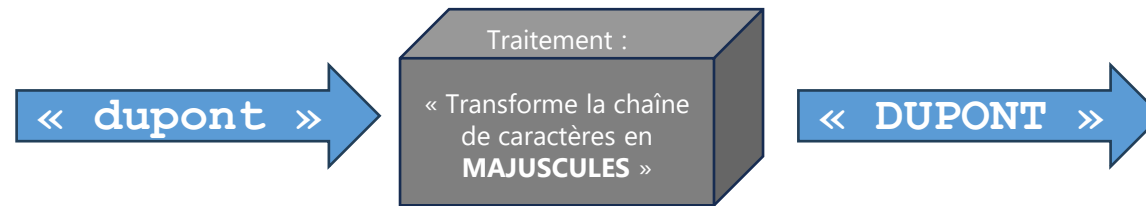
- **Techniques basées sur l'expérience** :

Elles se fondent sur l'expérience des testeurs et développeurs.

Ex: *Estimation d'erreur, tests exploratoires, tests basés sur les checklists*

Identifier les données d'entrée

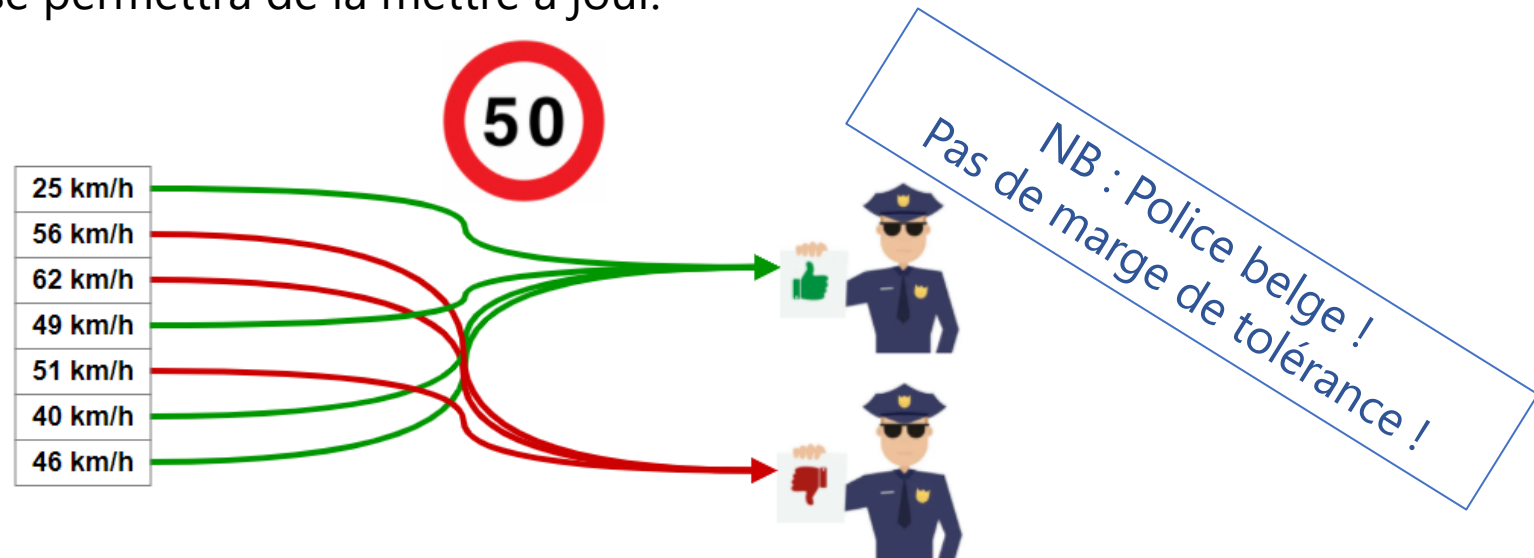
- **Les exigences définissent les règles que le traitement appliquera aux données en entrée.**
Ainsi, **choisir une donnée d'entrée** de test, auquel sera appliquée un traitement défini par des règles **permet de « prédire » les données en sortie**, le résultat attendu. Ce résultat attendu est aussi appelé **« oracle de test »**.



- Pour **concevoir les cas de test et préparer les jeux de données** qui permettront de **valoriser et exécuter les cas de test**, il est essentiel de procéder à une phase **d'analyse et d'identification des données** que l'on utilisera.
- Pour cela, on appliquera des **techniques visant à classer et trier les données** d'entrée, puis à **choisir** celles qui constitueront les **jeux de données de test**.
- **L'objectif est ici d'optimiser la couverture et la pertinence des cas de test.**

Test par partition d'équivalences :

- Une partition d'équivalence regroupe **l'ensemble des valeurs** d'un test qui sont censées produire un **même résultat**. Ces valeurs appartiennent à la même **classe d'équivalence**.
- Le comportement de l'application doit être vérifié pour chaque partition d'équivalence, en **choisissant une valeur quelconque dans la plage** couverte par la partition d'équivalence.
 - **Si** cette valeur conduit à une exécution en **succès**, **alors** il en est de même pour toutes les valeurs de la classe.
 - **Si** au contraire, une **erreur** de comportement doit survenir à l'exécution, **alors** n'importe quelle valeur de la classe permettra de la mettre à jour.



Les règles de partitionnement :

- Si une condition d'entrée ou de sortie définit **un ensemble de valeurs** (ex : une énumération de valeurs) il faut prendre :
 - Une classe d'équivalence **valide** = représentant les valeurs incluses dans l'ensemble prédéfini ;
 - Une classe d'équivalence **invalid** = représentant les valeurs hors de l'ensemble.

Ex : Dans un formulaire, le champ « Civilité » accepte uniquement les valeurs « Monsieur » et « Madame ».

- Classe d'équivalence **valide** :

{ Monsieur; Madame }

- Classe d'équivalence **invalid** :

Toute autre chaîne de caractères. Par exemple, « Mademoiselle » ou « Mme ».

Les règles de partitionnement (2/2) :

- Si une condition d'entrée ou une condition de sortie définit une contrainte devant être vérifiée (ex : « Le premier des 5 caractères de l'identifiant doit être une lettre. ») il faut prendre :
 - Une classe d'équivalence **valide** = la condition est vérifiée ;
 - Une classe d'équivalence **invalid** = la condition n'est pas vérifiée.

Ex : Classe d'équivalence **valide** : A2345, ZB678 ...

Classe d'équivalence **invalid** : 23456, 2B345 ...

- ATTENTION ! Si une classe d'équivalence semble **trop complexe** ou si les classes d'équivalence semble posséder des **éléments traités différemment** par le système il faut :
 - **Décomposer** cette classe d'équivalence complexe en **plusieurs classes d'équivalence moins complexes**.

Exemple 1 : Soit la **règle de gestion (RG)** suivante : « L'âge doit être compris entre 18 et 25 ans. » :

Cette règle de gestion implique trois partitions d'équivalence :

1. Partition d'équivalence **valide** : $18 \leq \text{âge} \leq 25$.
2. Partition d'équivalence **invalid** n°1 : $\text{âge} < 18$.
3. Partition d'équivalence **invalid** n°2 : $\text{âge} > 25$.

Ces trois partitions d'équivalence devront être testées, par exemple en effectuant des tests avec les valeurs « 20 », « 12 », « 37 ».

Exemple 2 : **RG** : « Le numéro de département doit être compris entre 1 et 95 »

Validité des entrées	Classes d'équivalence	Données de test
Entrées valides	[1 – 95]	« 11 »
Entrées invalides (basses)	[minInt – 1 [	« -30 », « 0 »
Entrées invalides (hautes)	] 95 – MaxInt]	« 100 »



Comprendre les classes d'équivalence

On cherche à tester un système :

Logiciel de reconnaissance de formes à partir d'un capteur vidéo. On nous demande de tester sa capacité à identifier formes, couleurs et taille.

Pour le tester, on dispose d'un jeu d'inputs, se présentant comme un ensemble de pastilles, de formes, tailles, et couleurs différentes.

Pour organiser nos tests, on vous demande de :

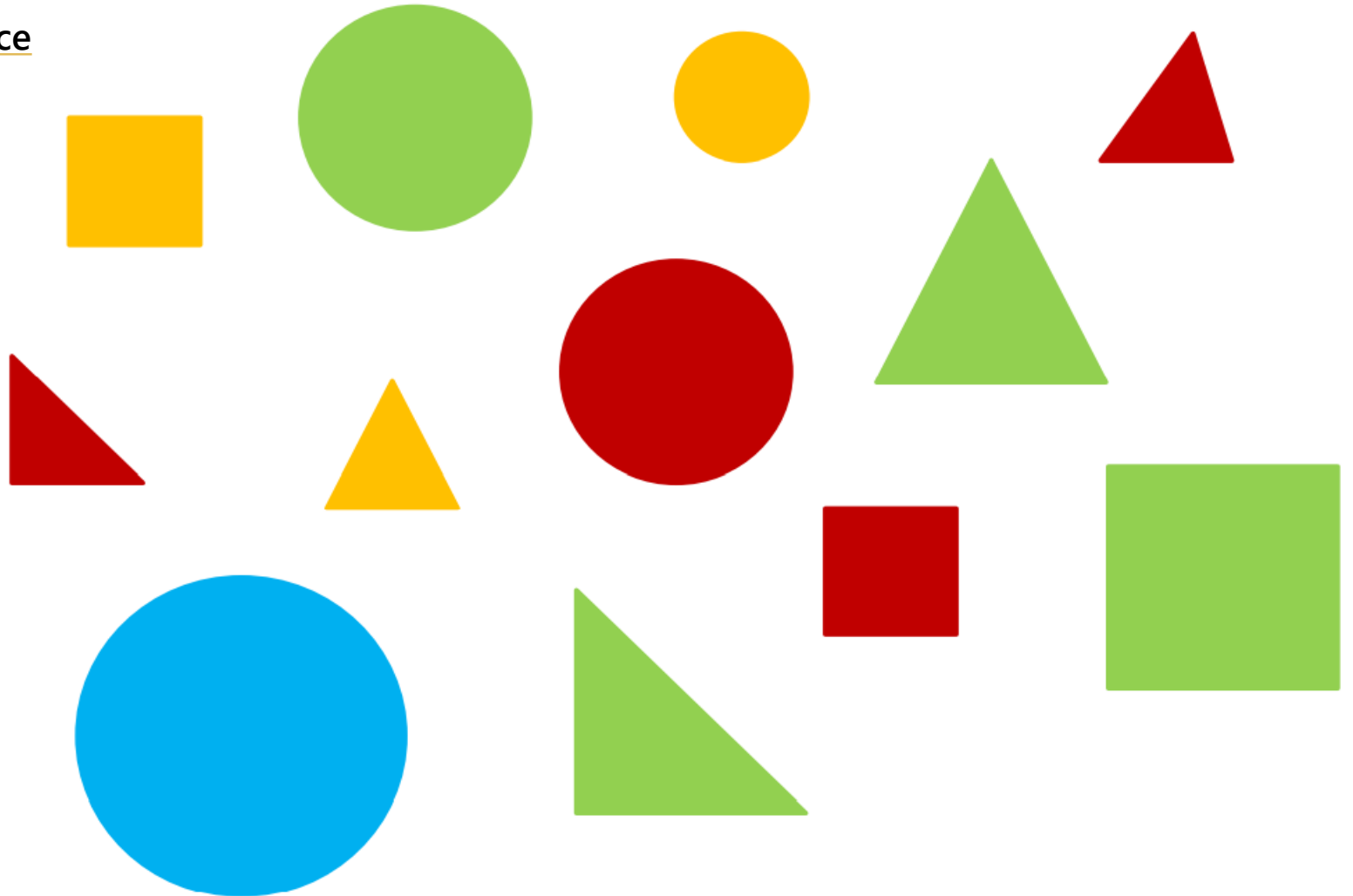
1. Déterminer les critères à tester
2. Organiser les pastilles par classes d'équivalences selon ces critères dans des tableaux
3. Créer une classification arborescente qui déterminera les différents cas de tests à exécuter
4. Déterminer le nombre minimal de cas de tests apportant la couverture la plus complète



Comprendre les classes d'équivalence

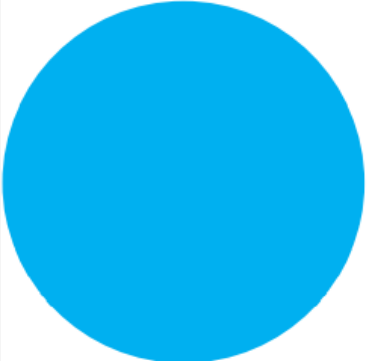
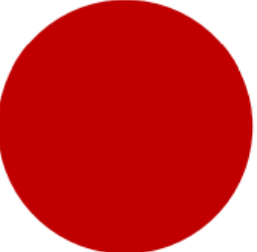
Le jeu de pastilles :

Pour déterminer les classes d'équivalence, on se posera la question :
« *Comment peut-on les trier ?* »



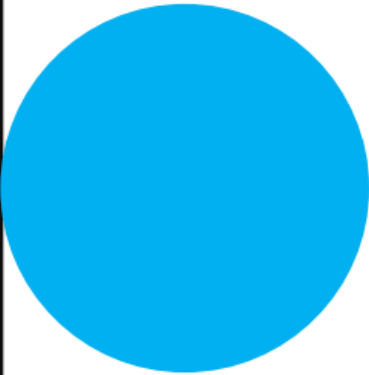
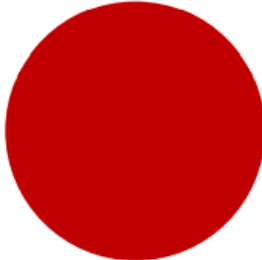
Comprendre les classes d'équivalence

Classement par taille

Grand	Moyen	Petit
	    	     

Comprendre les classes d'équivalence

Classement par couleur

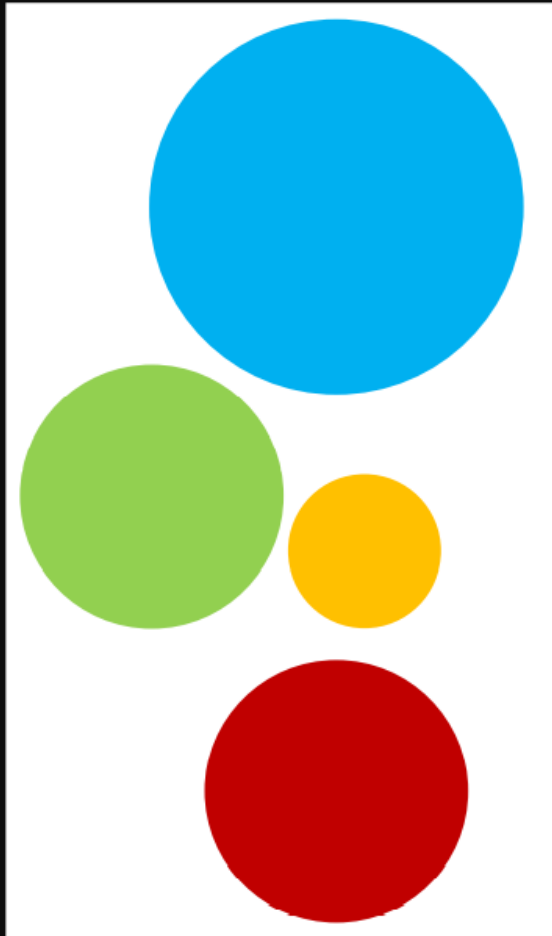
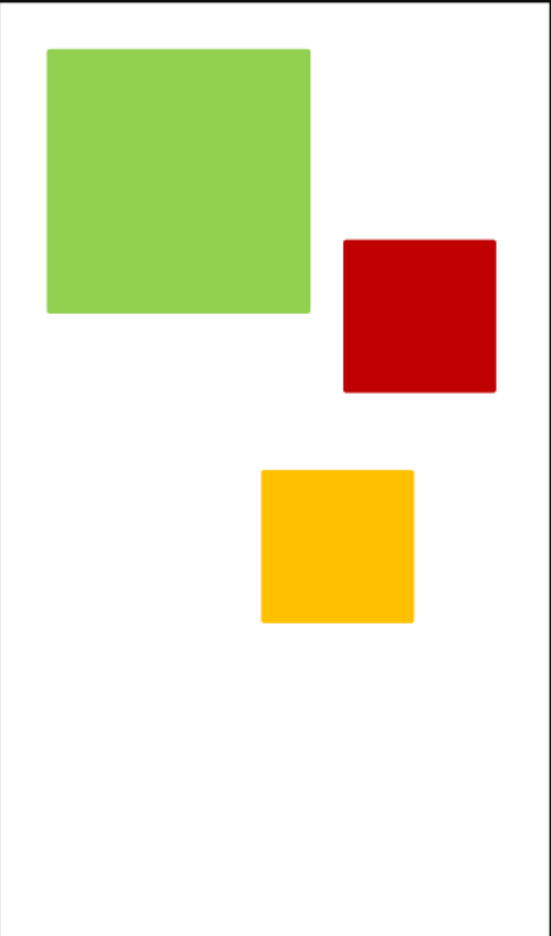
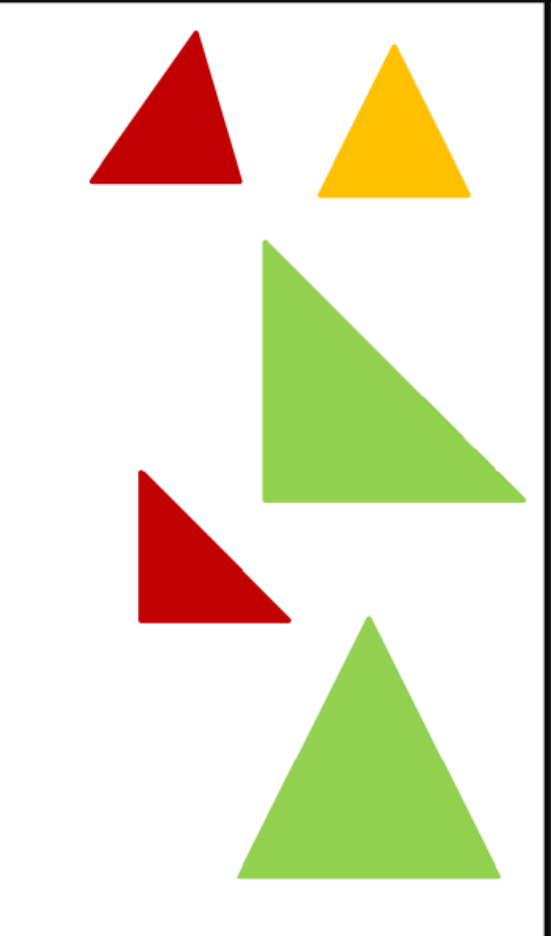
Bleu	Vert	Rouge	Jaune
	   	   	  



Comprendre les classes d'équivalence

Classement par forme

Mais ... peut-on affiner ?

Cercle	Carré	Triangle
		



Comprendre les classes d'équivalence

Classement par forme de triangle (branche fille du classement par forme)

Équilatéral	Scalène *	Rectangle
 		 

* Triangle scalène : triangle dont les 3 côtés sont de longueurs différentes.



Comprendre les classes d'équivalence

Représenter le premier niveau de classification sous forme d'arborescence.

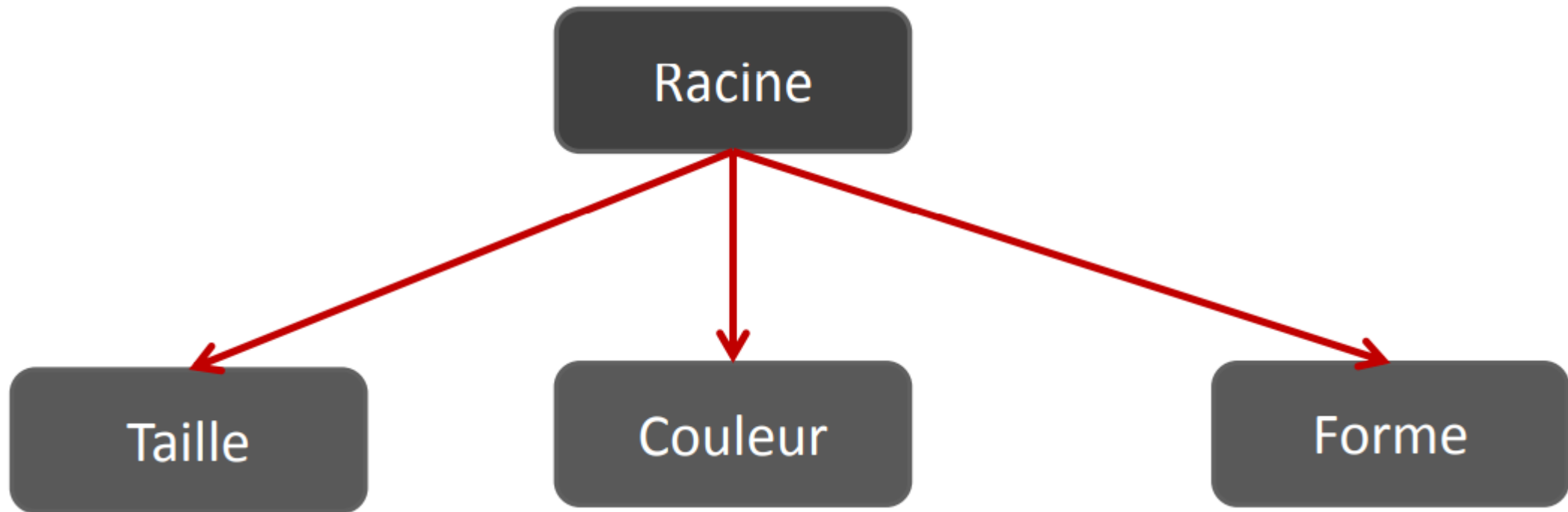


Pensez au Mindmapping ;-)



Comprendre les classes d'équivalence

Représenter le premier niveau de classification sous forme d'arborescence.





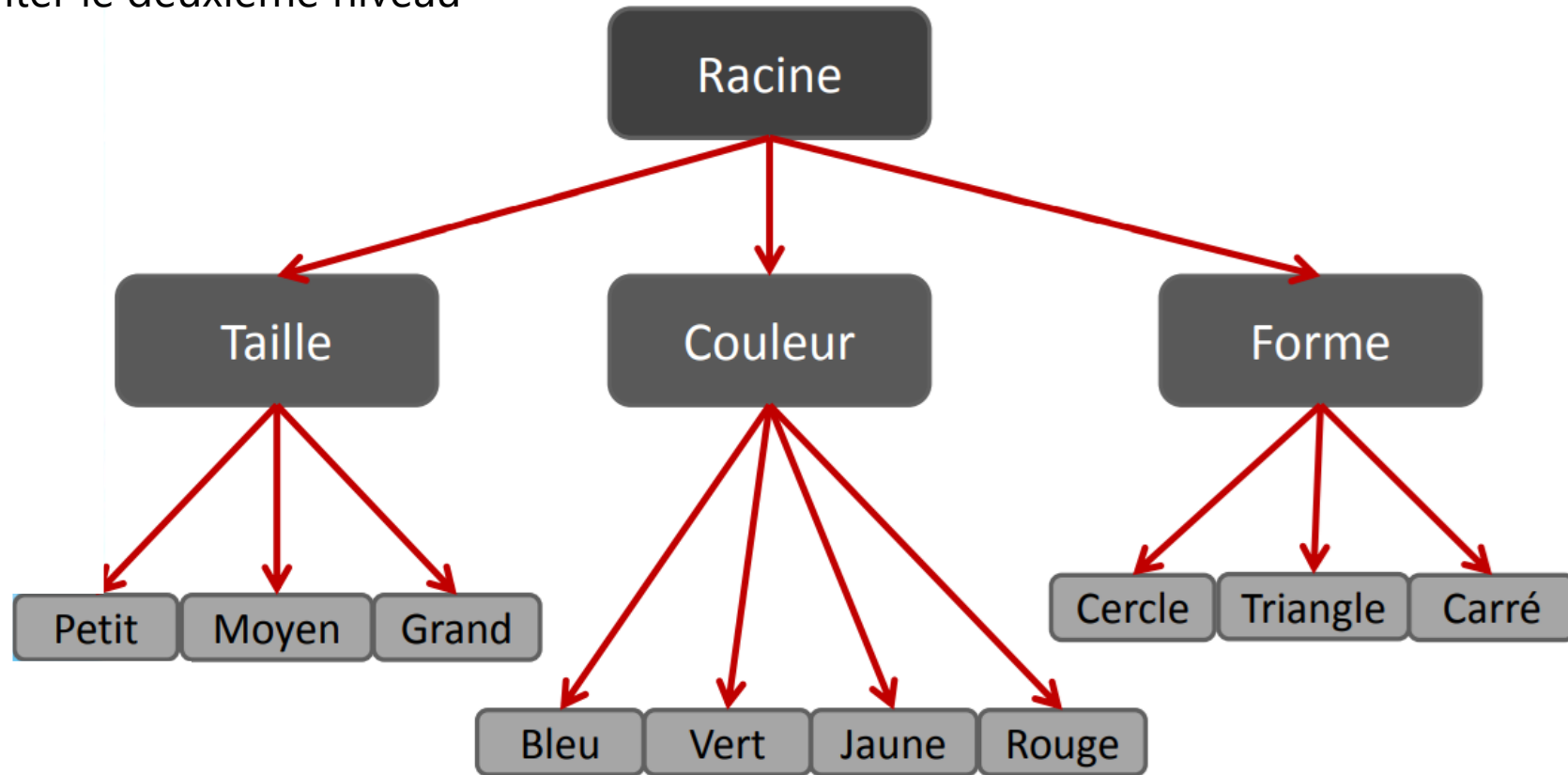
Comprendre les classes d'équivalence

Représenter le deuxième niveau



Comprendre les classes d'équivalence

Représenter le deuxième niveau





Comprendre les classes d'équivalence

Représenter le troisième niveau

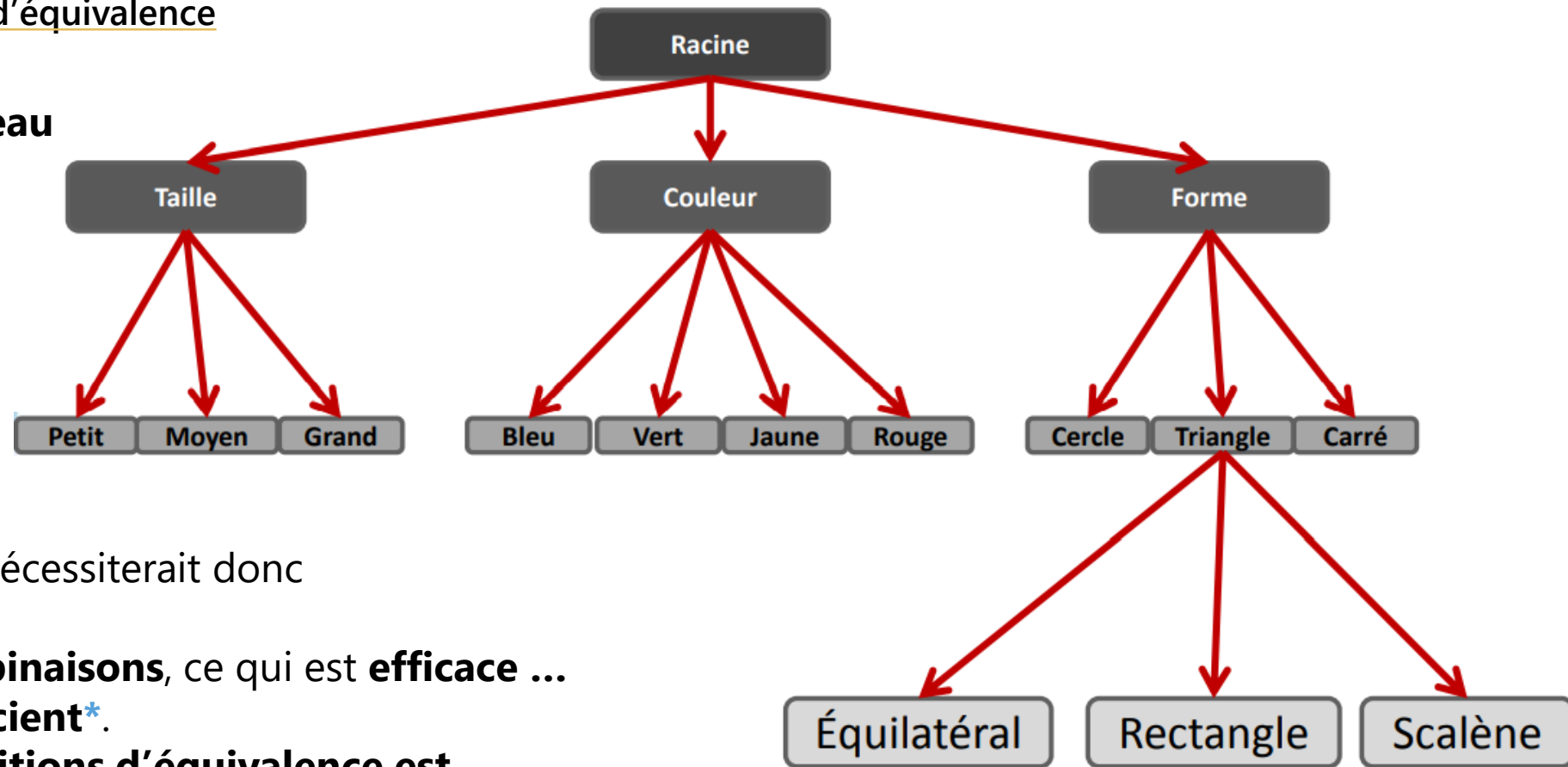


Comprendre les classes d'équivalence

Représenter le troisième niveau

On a :

- 5 formes
- 4 couleurs
- 3 tailles



Tester de manière exhaustive nécessiterait donc

5 x 4 x 3 = 60 cas de tests.

On teste ainsi **toutes les combinaisons**, ce qui est **efficace** ...
mais peut-être pas le plus **efficace***

Cependant, **l'analyse des partitions d'équivalence est un préalable obligatoire** à toutes les techniques d'optimisation que nous verrons par la suite.

* Nous verrons plus loin comment optimiser l'effort de test

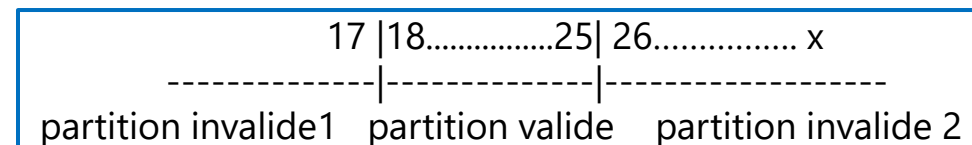
Test aux limites :

- **L'analyse aux limites** est une **extension des partitions d'équivalence**, mais ne peut être utilisée **que lorsque la partition est ordonnée**, composée de données numériques ou séquentielles.
- Les valeurs **minimale et maximale** (ou première et dernière valeurs) d'une partition sont ses **valeurs limites**.
- Le **test aux limites** consistera à étudier le comportement du système avec ces valeurs.
- En fonction du contexte (et du temps accordé) on **étudiera pour chaque limite 2 ou 3 valeurs**.

Exemple 1 : Soit l'exigence suivante à tester : « L'âge doit être compris entre 18 et 25 ans »

On a là 3 partitions :

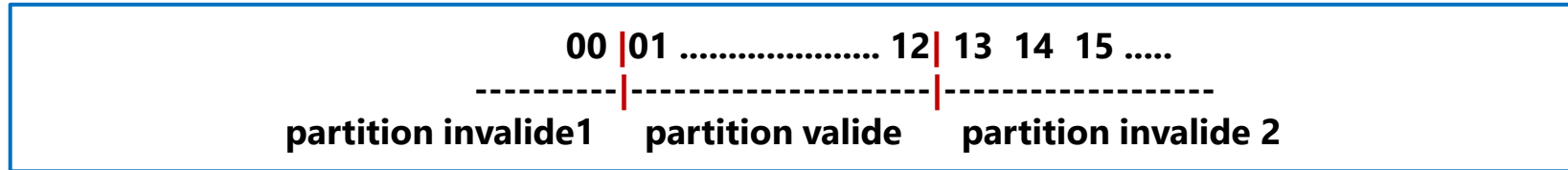
- 1 à 17 ans => **Partition invalide basse**
- 18 à 25 ans => **Partition valide**
- 26 à x ans => **Partition invalide haute**



Un **test aux limites** devra permettre de vérifier les valeurs suivantes :

- **(17, 18, 25, 26)** si on choisit de tester **2 valeurs par limite**.
- En choisissant de tester **3 valeurs par limite**, on vérifierait **(17, 18, 19, 24, 25, 26)**

Exemple 2 : Les valeurs attendues sont des mois calendaires, saisis sous forme d'entiers à 2 digits, les **partitions** seront les suivantes (ensemble ordonné) :



Un test aux limites devra prendre en compte les **valeurs** suivantes : (00, 01, 12, 13) **ou bien** (00, 01, 02, 11, 12, 13)

Exemple 3 : Un champ d'entrée accepte une seule valeur entière comme entrée, en **utilisant un pavé numérique** qui limite les entrées. **La plage valide va de 1 à 5 inclus.**

Partition invalide basse : 0 (une seule valeur) **Partition valide** : (1, 2, 3, 4, 5) **Partition invalide haute** : (6, 7, 8, 9)

Un test aux limites devra prendre en compte les **valeurs** suivantes : (0, 1, 5, 6) **ou bien** (0, 1, 4, 5, 6)

Le comportement aux limites des partitions d'équivalence est plus susceptible d'être incorrect que le comportement à l'intérieur des partitions.

Lors du développement, les limites spécifiées et implémentées peuvent être décalées vers des positions supérieures ou inférieures à leurs valeurs prévues, être oubliées, incomprises ou être complétées avec des limites supplémentaires non souhaitées.



Quelles sont les valeurs à sélectionner dans ces 3 cas pour un test aux limites ?

(Utiliser les techniques « 2 valeurs » et « 3 valeurs »)

- Pour l'intervalle « de l'année 2000 à l'année 2016 »;
- Pour l'ensemble ordonné de valeurs, représentant les mois de janvier à décembre, représentés par un nombre au format « mm »;
- Pour la condition d'entrée : « Le champ civilité doit être renseigné avec une de ces trois valeurs: { 'Monsieur', 'Madame', 'Docteur' }.



En pratique, il est intéressant de matérialiser les données sur une ligne comportant des intervalles ouverts ou fermés représentés par des crochets ([])

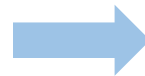


Quelles sont les valeurs à sélectionner pour ces 3 cas pour un test aux limites ?

- **Pour l'intervalle « de l'année 2000 à l'année 2016 » :**

Valeurs **valides** : **borne** « 2000 » ; **borne** « 2016 ».

Valeurs **invalides** : « 1999 » et « 2017 ».



Valeurs retenues pour des tests aux limites :
(1999,2000,2016,2017)

- **Pour l'ensemble ordonné de valeurs :**

Mois de janvier à décembre, au format MM

Valeurs retenues pour des tests aux limites : {00,01,12,13}

- **Pour la condition d'entrée : « Le champ civilité doit être renseigné avec une valeur parmi les valeurs 'Monsieur', 'Madame' et 'Docteur'. » :**

Nombre minimum de valeurs dans les limites : 1

Nombre maximum de valeurs dans les limites : 3

Nombre de valeurs hors limites : 2 (aucune valeur et une valeur 'autre').



Étudier le cas suivant, dans une optique de test aux limites ?

Soit la fonctionnalité de calcul de frais d'expédition en fonction du poids d'un colis.

- Cette fonction prend le poids en entrée et retourne le coût d'expédition correspondant.
- Les poids valides pour les colis se situent entre un minimum de 0,5 kg et un maximum de 10 kg.
- Le coût d'expédition varie en fonction du poids :
 - De 0,5 kg (inclus) à 2 kg (exclus) : 5 €
 - De 2 kg (inclus) à 5 kg (exclus) : 8 €
 - De 5 kg (inclus) à 10 kg (inclus) : 12 €



Étudier le cas suivant, dans une optique de test aux limites ?

Pour effectuer un test aux limites, on pourrait prendre les valeurs limites de chaque intervalle ainsi que les valeurs juste en dehors de ces intervalles pour s'assurer que la fonction gère correctement les cas limites.

Voici quelques exemples de scénarios de test aux limites, comportant **7 cas de test** :

- Test avec le **poids minimum** (0,5 kg) : On s'attend à ce que le coût soit de 5 €.
- Test avec un **poids supérieur au maximum** (10,01 kg) : On s'attend à ce que le coût soit 0 € ou qu'une erreur soit signalée, selon les Spécifications, car le poids est en dehors de la plage valide.
- Test avec un **poids à la limite inférieure de chaque intervalle** (2 kg et 5 kg)
On s'attend respectivement à des coûts de 5 € et 8 €.
- Test avec un **poids à la limite supérieure de chaque intervalle** (1,99 kg, 4,99 kg et 9,99 kg) : On s'attend respectivement à des coûts de 5 €, 8 € et 12 €.

Ces scénarios de test aux limites permettent de vérifier si la fonction de calcul des frais d'expédition gère correctement les valeurs limites et les cas où le poids est juste en dehors des intervalles valides.

- **Test par tables de décisions :**
 - Une **table de décision** est un **outil logique** permettant de **modéliser** facilement un ensemble de choix **d'une certaine complexité**. Au lieu d'obtenir une série de conditions imbriquées par une succession de SI..., ALORS..., SINON..., il est possible de créer un tableau représentant les conditions et les décisions en fonction des valeurs pertinentes.
 - La table de décision est utile pour identifier les cas à tester dans les **situations de dépendances entre partitions d'équivalence**.
 - Pour formaliser les tests et identifier les jeux de données nécessaires à la vérification de ces règles de gestion, on peut s'aider d'une **table de décision**.
 - **Les tables de décisions se construisent en deux temps :**
 1. **Identification** : On matérialise tous les « cas logiques » possibles.
 2. **Simplification** : On élimine les éléments redondants, non pertinents ou non essentiels de la table. Cela peut rendre la table plus claire et plus facile à gérer.

- **Test par tables de décisions :**

Par exemple, une table de décisions sera utile pour formaliser les **conditions d'obtention d'un prêt bancaire** suivantes :

- Si le client est solvable, et que son salaire est au moins de 2 000 €, il peut obtenir un prêt.
- Si son salaire est inférieur à 3 000 €, le montant du prêt ne peut pas dépasser 6 000 €
- Si son salaire est inférieur à 5 000 €, le montant du prêt ne peut pas dépasser 8 000 €
- Si son salaire est supérieur à 5 000 €, le montant du prêt ne peut pas dépasser 12 000 €
- Pour un prêt inférieur à 6 000 €, le taux est de 1,5% et la mensualité minimale est de 300 €
- Pour un prêt inférieur à 8 000 €, le taux est de 1,3% et la mensualité minimale est de 400 €
- Pour un prêt inférieur ou égal à 12 000 €, le taux est de 1% et la mensualité minimale est de 600 €

Problématique :

Quelle décision sera prise par le système de la banque pour accorder ou non un prêt à un client, en déterminant le taux ainsi que les mensualités minimales ?

(Solution dans 2 slides)

- **Construction d'une table de décisions :**

Une table de décision peut être construite de différentes manières mais une façon simple de procéder consiste à créer un tableau avec :

En abscisse (lignes) :

1. Identifier les **conditions** qui peuvent avoir une influence sur les décisions (le résultat du traitement). On indiquera dans les colonnes les cas où la condition est **VRAIE (V)** et les cas où elle est **FAUSSE (F)**.

Dans une table de décisions, **lorsque certaines conditions ne sont pas applicables à un cas particulier**, vous pouvez les indiquer en utilisant la **notation « 0 » (zéro) "N/A" (non applicable), ou "—" (tiret) pour montrer qu'elles ne sont pas pertinentes dans ce contexte**. Cela permet de clarifier que certaines conditions ne s'appliquent pas à certains cas et d'éviter toute confusion lors de la lecture de la table.

2. Identifier les **actions** résultantes. Pour chaque action, on fera figurer dans les colonnes du tableau les cas où le résultat s'avère (**VRAI**) et ceux où il ne se produit pas (**FAUX**).

En ordonnées (colonnes) :

Les cas de tests en identifiant quelle action doit être produite en réponse à une combinaison de décisions particulière.

Puis enfin on pourra :

3. Vérifier que la table est **complète** (toutes les actions sont au moins exercées une fois) et **consistante** (pour un même variant, deux actions incompatibles ne sont pas exercées simultanément).

4. **Simplifier la table** pour optimiser l'effort de test.

Technique de conception des tests : tables de décisions

- Si le client est solvable, et que son salaire est au moins de 2 000 €, il peut obtenir un prêt.
- Si son salaire est inférieur à 3 000 €, le montant du prêt ne peut pas dépasser 6 000 €
- Si son salaire est inférieur à 5 000 €, le montant du prêt ne peut pas dépasser 8 000 €
- Si son salaire est supérieur à 5 000 €, le montant du prêt ne peut pas dépasser 12 000 €
- Pour un prêt inférieur à 6 000 €, le taux est de 1,5% et la mensualité minimale est de 300 €
- Pour un prêt inférieur à 8 000 €, le taux est de 1,3% et la mensualité minimale est de 400 €
- Pour un prêt inférieur ou égal à 12 000 €, le taux est de 1% et la mensualité minimale est de 600 €

Condition

Cas de test

		CAS DE TEST DÉDUITS						
	CONDITIONS	Cas de test 1	Cas de test 2	Cas de test 3	Cas de test 4	Cas de test 5	Cas de test 6	Cas de test 7
Condition 1	Client solvable ?	O	O	O	O	O	O	N
Condition 2	Salaire	>= 2 000 € < 3 000 €	>= 2 000 € < 3 000 €	>= 3 000 € < 5 000 €	>= 3 000 € < 5 000 €	>= 5 000 €	>= 5 000 €	---
Condition 3	Montant demandé	<= 6 000€	> 6 000 €	<= 8 000 €	> 8 000 €	<= 12 000 €	> 12 000 €	---
ACTIONS RÉSULTANTES								
Action 1	Accord du prêt	OK	KO	OK	KO	OK	KO	KO
Action 2	Mensualité minimale	300 €	---	400 €	---	600 €	---	---
Action 3	Taux d'intérêt applicable	1,50%	---	1,30%	---	1%	---	---

Action
résultante



Construisons une table de décisions (exercice en « pas à pas »)

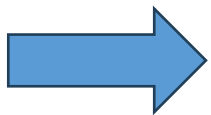
On doit tester les conditions de ventes pour un site de librairie en ligne :

Conditions générales

- Le paiement peut être effectué à la commande dans tous les cas de figure.
- Le paiement à la livraison est possible dans certains cas.
- Un acompte de 15% à la commande peut être demandé.

Conditions particulières

- Un paiement à la commande est exigé pour tout nouveau client.
- Les clients ayant effectué une commande **supérieure ou égale** à 1 000 € dans les 30 jours précédents ont le choix de régler à la commande ou à la livraison. Si le client choisit alors de régler à la réception, et si la transaction est **supérieure ou égale** à 500 €, un acompte de 15% à la commande sera exigé.



À partir de ces règles de gestion, déterminez les conditions et les actions possibles.



Construisez la table de décisions (exercice en « pas à pas »)



Conditions	C1 : Être (ou pas) nouveau client.	V ou F ou 0
	C2 : Avoir (ou pas) effectué en M-1 une commande supérieure ou égale à 1000 €	V ou F ou 0
	C3 : Réaliser une transaction $\geq 500\text{€}$	V ou F ou 0
Actions	A1 : Acompte 15% à la commande exigé	V ou F ou 0
	A2 : Paiement à la commande exigé/possible	V ou F ou 0
	A3 : Paiement à la livraison possible	V ou F ou 0

Commencez par étudier le cas « Nouveau client »



Construisez la table de décisions (exercice en « pas à pas »)

		CT 1
Conditions	C1 : Nouveau client.	V
	C2 : M-1 >= 1000 €	0
	C3 : Transaction >= 500€	V
Actions	A1 : Acompte 15%	F
	A2 : Paiement à la commande	V
	A3 : Paiement à la livraison	F

Conditions

Un « Nouveau client » (Cas de Test 1) :

- Est par définition Nouveau => V
- N'a pas de transaction le mois précédent => 0
- Effectue une transaction d'un montant supérieur ou égal à 500 € => V

Actions
résultantes

- Doit verser un acompte => F
- Peut/doit payer à la commande => V
- Peut/doit payer à la réception => F



Construisez la table de décisions (exercice en « pas à pas »)

		CT 1	CT 2
Conditions	C1 : Nouveau client.	V	V
	C2 : M-1 >= 1000 €	0	0
	C3 : Transaction >= 500€	V	F
Actions	A1 : Acompte 15%	F	F
	A2 : Paiement à la commande	V	V
	A3 : Paiement à la livraison	F	F

Conditions

Autre possibilité pour un « Nouveau client » (CT2) :

- Est par définition Nouveau => **V**
- N'a pas de transaction le mois précédent => 0
- Effectue une transaction d'un montant supérieur ou inférieur à 500 € => **F**

Actions
résultantes

- Doit verser un acompte => **F**
- Peut/doit payer à la commande => **V**
- Peut/doit payer à la réception => **F**



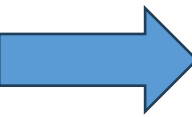
Construisez la table de décisions (exercice en « pas à pas »)

On peut donc poser notre « oracle de test » pour le cas d'un nouveau client, effectuant une transaction, et déduire à partir du tableau précédent le comportement du système (= le résultat attendu).
Il est même possible de prévoir les **données de test** nécessaires.

En transposant le tableau, on obtient les 2 cas de tests de haut niveau, valorisés :

Cas de test		Résultat attendu
CT1	Un nouveau client effectue une commande d'un montant de 500 €.	Seul le paiement à la commande lui est proposé.
CT2	Un nouveau client effectue une commande d'un montant de 499 €.	Seul le paiement à la commande lui est proposé.

NB : Ici, on en profite pour **étudier le système « aux limites »**, plus susceptibles de révéler des dysfonctionnements.
De plus, le traitement et les résultats attendus étant identiques dans les deux cas, **on pourrait se contenter d'un seul cas de test**, avec un montant de transaction quelconque. Cette option sera choisie en fonction de la criticité du système et de la stratégie de recette établie

 **La table de décisions peut maintenant être complétée pour les différents cas de figure concernant le « Client déjà enregistré » (donc, « Nouveau client » = F)**



Construisez la table de décisions (exercice en « pas à pas »)

	CT 1	CT 2	CT 3	CT4	CT5	CT 6
C1 : Nouveau client.	V	V	F	F	F	F
C2 : M-1 >= 1000 €	0	0	V	V	F	F
C3 : Transaction >= 500€	V	F	V	F	V	F

A1 : Acompte 15%	F	F	V	F	F	F
A2 : Paiement à la commande	V	V	V	V	V	V
A3 : Paiement à la livraison	F	F	V	V	F	F

Le tableau est maintenant complété.

On peut en déduire tous les cas de tests de haut niveau et les valoriser.

Ensuite, on tentera **d'optimiser** :

1. Y a-t-il des CT peu pertinents ou signifiants, qui seraient moins prioritaires ?
2. Y a-t-il des cas alternatifs qu'il serait intéressant d'étudier ?



Construisez la table de décisions (exercice en « pas à pas »)

Cas de test	Résultat attendu
CT1 : Un nouveau client effectue une commande d'un montant de 500 €.	Seul le paiement à la commande lui est proposé.
CT2 : Un nouveau client effectue une commande d'un montant de 499 €.	Seul le paiement à la commande lui est proposé.
CT3 : Un client connu effectue une commande d'un montant de 500 €. Le mois précédent, il a effectué une commande de 1000 €	Le paiement à la commande ou à la livraison lui est proposé. Un acompte de 75 € lui est demandé à la commande.
CT4 : Un client connu effectue une commande d'un montant de 499 €. Le mois précédent, il a effectué une commande de 1000 €	Le paiement à la commande ou à la livraison lui est proposé. Aucun acompte ne lui est demandé.
CT5 : Un client connu effectue une commande d'un montant de 500 €. Le mois précédent, il a effectué une commande de 999 €	Le paiement à la commande seul lui est proposé. Aucun acompte ne lui est demandé.
CT6 : Un client connu effectue une commande d'un montant de 499 €. Le mois précédent, il a effectué une commande de 999 €	Le paiement à la commande seul lui est proposé. Aucun acompte ne lui est demandé.

Avec 6 cas de tests toutes les décisions sont couvertes.

Cas supplémentaire possible :

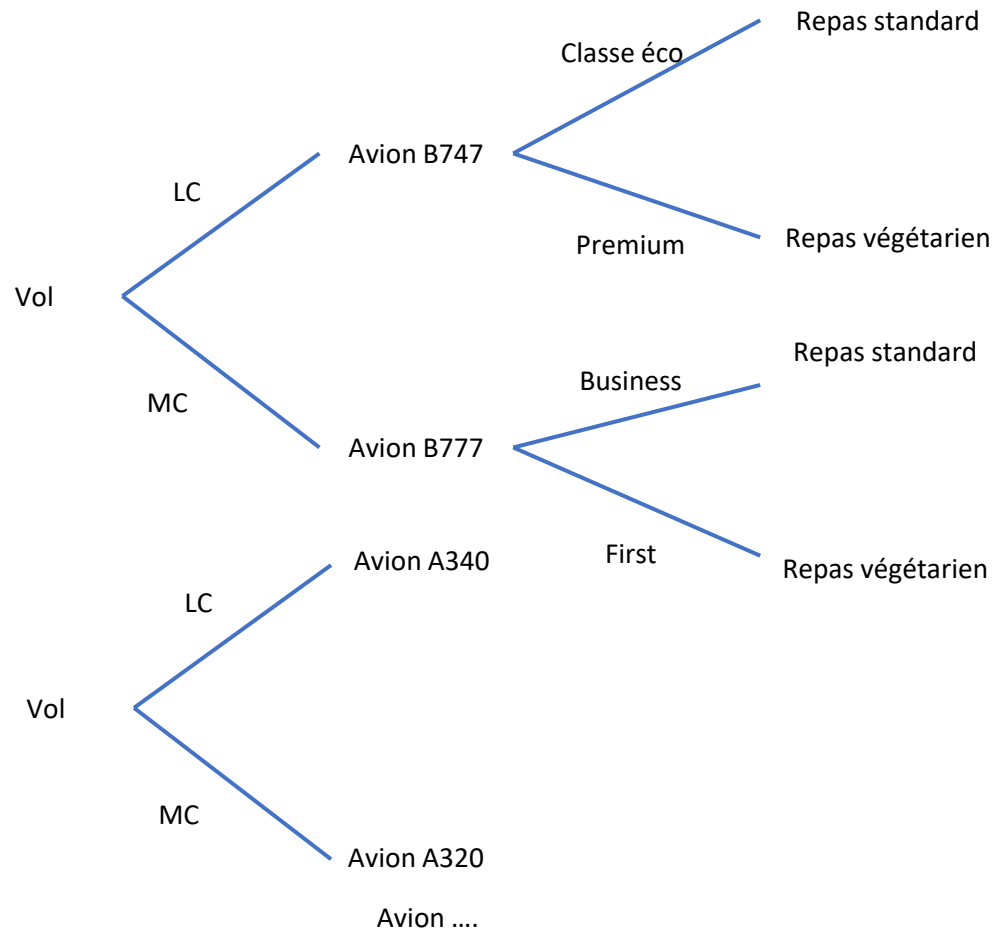
Un client connu du système effectue une commande. Le mois précédent, il n'a passé aucune commande ($M-1 = 0$ €)

Priorisation des CT :

CT1 et CT2 ont le même résultat, de même que CT5 et CT6. On pourrait simplifier le tableau en supprimant un CT pour chaque paire.

Parfois, la technique des tableaux de décisions peut ne pas être suffisante, si la complexité combinatoire augmente fortement.

En reprenant les données de l'exercice « Altea » (Édition de cartes d'embarquement pour une compagnie aérienne)



- Avec 6 types d'avions, 4 classes, 3 types de repas, et les distinctions adulte/enfant, handicapé/valide, groupe/individuel, retard/à l'heure, il existe **1152 combinaisons possibles** ($6 \times 4 \times 3 \times 2 \times 2 \times 2 \times 2$) des valeurs des 7 paramètres ().
- Tester ces 1152 combinaisons permettrait de **garantir l'exhaustivité de la couverture** des tests.
- **NB :**
 - Il s'agit ici d'un exemple simple. La complexité combinatoire peut être importante :
13 paramètres de 3 valeurs $\Rightarrow 3^{13}$ possibilités, soit 1 594 323 (1,6 million) cas de tests.
 - **20 paramètres de 10 valeurs** $\Rightarrow 10^{20}$, soit $10^9 \times 10^9 \times 10^2$, soit 100 milliards de milliards de cas de test. Cet exemple pourrait correspondre à un formulaire de 20 champs comportant chacun 10 possibilités. Cas impossible ?

Technique de conception des tests : Traitement combinatoire (Pairwise)

- Lorsque la combinatoire est trop importante, on peut recourir à la technique dite « Pairwise », voire N-Wise (« All triplets » : 3-Wise; « All quadruplets » : 4-Wise)
- La technique de « Pairwise » / « par paire » se fonde sur la création d'un tableau orthogonal de **vecteurs de tests** composés par l'attribution d'une valeur à chaque paramètre.
- L'ensemble des vecteurs de test garantit que :
 - **Chaque valeur significative** est testée au moins une fois,
 - **Chaque paire de valeurs** est testée au moins une fois.
- **Exemple** : Je veux tester la tarte aux fruits
 - Mes paramètres sont :
 - La pâte : brisée ou feuilletée (2)
 - Les fruits de saison : fraise, cerise, abricot (3)
 - La crème : pâtissière ou confiture (2)
 - **2 x 3 x 2 = 12 combinaisons et cas de test possibles**
 - **All singles : 3 cas de test**
 - **All pairs : 6 cas de test**
 - **All triplets : 12 cas de test**

Cas de test	Pâte	Fruit	Crème
CT 1	Brisée	Fraise	Pâtissière
CT 2	Brisée	Abricot	Confiture
CT 3	Feuilleté	Cerise	Confiture

Tableau « All singles »

Cas de test	Pâte	Fruit	Crème
CT 1	Brisée	Fraise	Pâtissière
CT 2	Brisée	Abricot	Confiture
CT 3	Brisée	Cerise	Pâtissière
CT 4	Feuilletée	Fraise	Confiture
CT 5	Feuilletée	Abricot	Pâtissière
CT 6	Feuilletée	Cerise	Confiture

Tableau « All pairs »

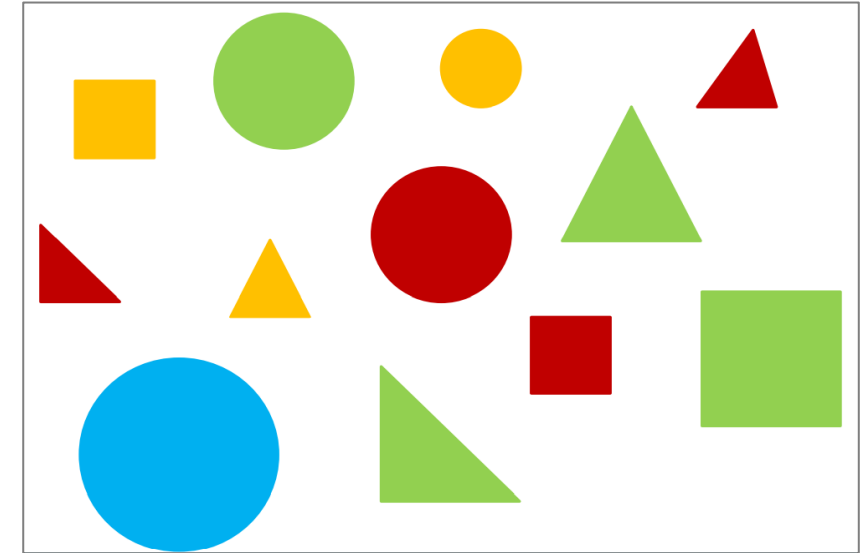


All singles / All pairs

Reprenons l'exemple de test du logiciel vidéo de reconnaissance de formes, vu plus tôt.

Avec **5 formes, 4 couleurs et 3 tailles**, nous avons conclu que **60 cas de test** permettaient de tester **exhaustivement** le système, c'est-à-dire toutes les combinaisons.

En utilisant la technique de test combinatoire, on pourra optimiser l'exécution/la charge de test en recourant aux tests « All singles » et « All Pairs ».



All singles : On s'assurera que chaque valeur de chaque paramètre est testée au moins 1 fois.



Si certaines anomalies se révèlent en utilisant une valeur particulière, de nombreuses autres ne se produisent **qu'en présence d'une combinaison de deux valeurs** simultanément utilisées. Plus « économique », le « **All singles** » **peut ne pas être suffisamment précis**.



Construire un tableau All Singles

- Un **tableau** permettra :
- de s’assurer que **chaque caractéristique sera prise en compte** dans les cas de test
 - de **déterminer le nombre minimal de cas de test** à réaliser

Démarche :

1. Déterminer pour chaque paramètre le **nombre de valeurs à étudier** :
 - Taille : 3
 - Couleur : 4
 - Forme : 5
2. Construire les **colonnes en commençant par les paramètres comportant le plus grand nombre de valeurs** (Ici : Forme > Couleur > Taille)

Cas de test	FORMES					COULEURS				TAILLE		
	Triangle			Carré	Cercle	Bleu	Vert	Jaune	Rouge	Petit	Moyen	Grand
	Équilatéral	Scalène	Rectangle									



Construire un tableau All Singles

- 3. Pour chaque paramètre, s’assurer de cocher 1 fois chaque valeur (X)
- 4. Chaque cas de test devant comporter 3 paramètres, compléter les cas de test en choisissant librement des valeurs déjà utilisées au pas précédent (X)

Cas de test	FORMES					COULEURS				TAILLE		
	Triangle			Carré	Cercle	Bleu	Vert	Jaune	Rouge	Petit	Moyen	Grand
	Équilatéral	Scalène	Rectangle									
CT 01	X					X				X		
CT 02		X					X				X	
CT03			X					X				X
CT04				X					X		X	
CT05					X		X					X

Conclusion : Avec **5 cas de test**, on couvre au moins 1 fois toutes les valeurs possibles, au lieu de **60 cas de tests** si on avait voulu tester de manière exhaustive (5 formes X 4 couleurs X 3 tailles) ...

Le gain est important, mais au détriment de la qualité. Si parfois ce niveau de qualité est suffisant, mieux vaut donc recourir au « All Pairs ».

Rappel : Les paramètres et valeurs retenus ne doivent pas montrer de dépendances entre eux !



Outiller le PairWise

Plus la combinatoire augmente, plus le gain lié au Pairwise devient important. Mais la création des tableaux se complexifie ...

Il est souvent nécessaire de l'outiller ! Par exemple, avec PICT ([outil Microsoft gratuit et open-source](#)) ou via des services en ligne ([souvent basés sur un logiciel existant - ici, PICT](#))

1-Téléchargez le fichier pict.exe, placez-le à la racine de C:\

PICT est un outil en ligne de commande, qui interprète un fichier .txt contenant le nom et les valeurs des paramètres retenus.

2-Avec Notepad++, créez le fichier pastilles.txt contenant le texte suivant (Ctrl-C, Ctrl-V) :

Formes:Tr_equilateral,TR_Scalene,TR_Rectangle

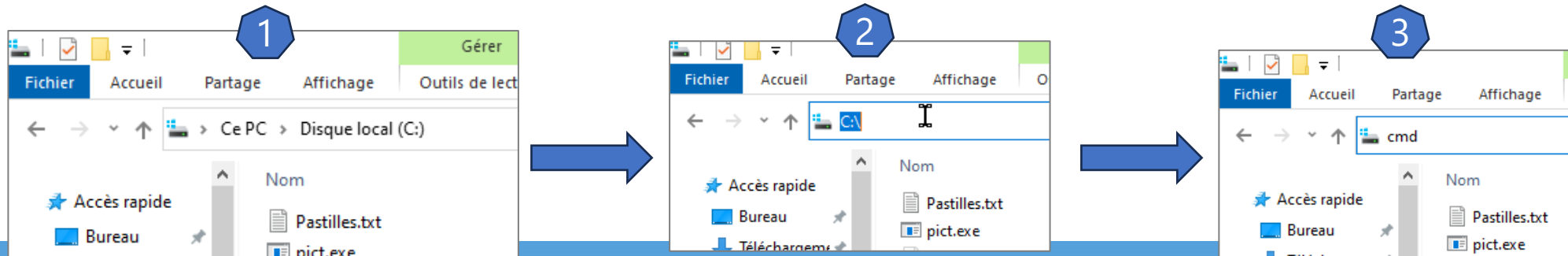
Couleurs:Bleu,Vert,Jaune,Rouge

Taille:Petit,Moyen,Grand

3-Enregistrez-le à la racine de C:\ et gardez l'Explorateur de fichiers ouvert.

4-Nous allons ouvrir l'invite de commandes Windows :

Cliquez dans la barre en haut de l'Explorateur de fichiers et tapez « cmd » puis touche Enter





Outiller le Pairwise

5- La fenêtre d'invite de commandes s'ouvre sur C:\>

6- À l'invite, tapez « pict.exe pastilles.txt » puis touche Enter

Lance l'exécution de PICT

Fichier source de données

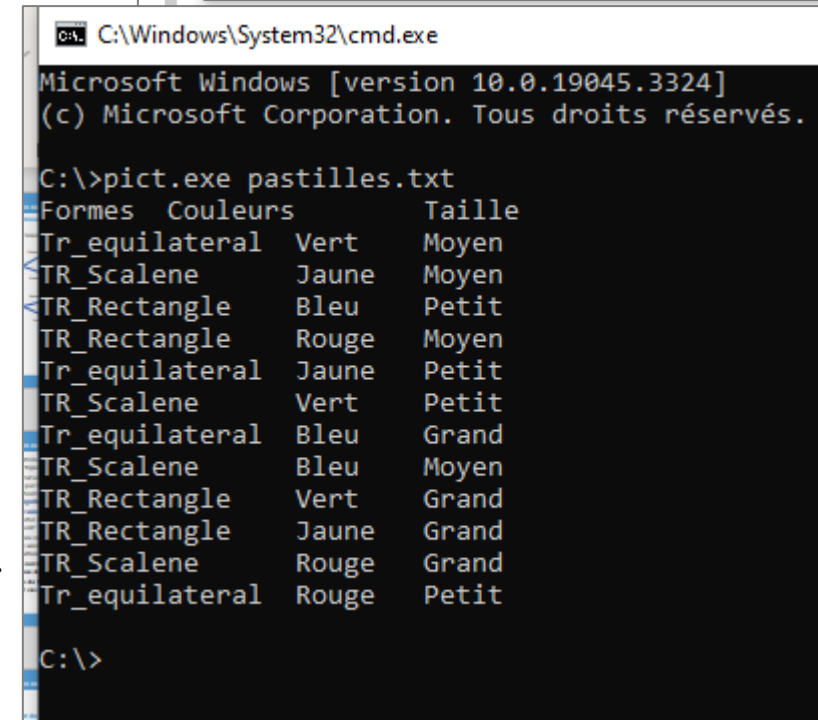
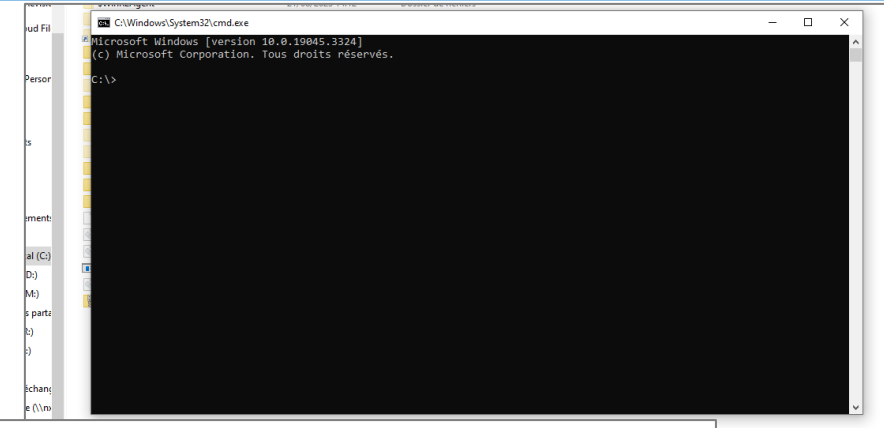
7- PICT affiche 12 vecteurs de test

Vous constatez que toutes les paires de valeurs de chaque paramètre sont présentes.

CONCLUSION :

Avec 12 cas de test en « All Pairs », la technique du Pairwise nous a permis d'optimiser la charge de test en maintenant une bonne couverture et une profondeur de test satisfaisante.

En savoir plus sur le Pairwise : www.pairwise.org





All singles / All pairs

Trois navigateurs (IE, Firefox, Chrome), deux OS (Windows, Linux) et quatre extensions (jpeg, png, gif et pdf) sont utilisés :

- All singles : Quels cas de test réaliser ?
- All pairs : Quels cas de test réaliser ?



All singles / All pairs

Trois navigateurs (IE, Firefox, Chrome), deux OS (Windows, Linux) et quatre extensions (jpeg, png, gif et pdf) sont utilisés :

- **All singles : Quels cas de test réaliser ?**
- All pairs : Quels cas de test réaliser ?

All singles

Cas de test	Extensions	Navigateurs	OS
CT 1	JPEG	IE	WINDOWS
CT 2	PNG	FIREFOX	LINUX
CT 3	GIF	CHROME	WINDOWS ou LINUX
CT 4	PDF	IE ou CHROME ou FIREFOX	LINUX ou WINDOWS

Chaque valeur de chaque paramètre est testée ici au moins une fois.

Mais beaucoup de choses restent à explorer :

Par exemple, quid de l'interaction JPEG/Chrome, ou des valeurs ignorées pour les CT 3 et 4 ?



All singles / All pairs

Trois navigateurs (IE, Firefox, Chrome), deux OS (Windows, Linux) et quatre extensions (jpeg, png, gif et pdf) sont utilisés :

- All singles : Quels cas de test réaliser ?
- **All pairs : Quels cas de test réaliser ?**

All pairs

Cas de test	Extensions	Navigateurs	OS
CT 1	JPEG	IE	WINDOWS
CT 2	JPEG	FIREFOX	LINUX
CT 3	JPEG	CHROME	WINDOWS
CT 4	PNG	IE	LINUX
CT 5	PNG	FIREFOX	WINDOWS
CT 6	PNG	CHROME	LINUX
CT 7	GIF	IE	A adapter en fonction de l'OS le plus utilisé
CT 8	GIF	FIREFOX	...
CT 9	GIF	CHROME	...
CT 10	PDF	IE	...
CT 11	PDF	FIREFOX	...
CT 12	PDF	CHROME	...



Sont explorées ici **toutes les paires de valeurs des interactions**

- Extension/Navigateur,
- Extension/OS
- Navigateur/OS

Pour quel gain ?

Au départ, nous avons **24** (4x3x2) vecteurs potentiels (exhaustifs)

Avec ce tableau, on a **divisé par 2** le nombre de cas de test.

Plus la combinatoire augmente, plus le gain lié au Pairwise devient important.

Il est souvent nécessaire de l'outiller !

Par exemple, avec PICT...



Exercice Pairwise

Formaliser une étude Pairwise avec PICT

- Pré-requis : Utiliser un service en ligne ou télécharger PICT et l'installer en stand-alone
- Reprendre les données de l'exemple précédent
- Créer un tableau Pairwise « simple », sans contrainte(s)
- (OPTIONNEL) Dans un deuxième temps, introduire les contraintes que vous jugerez utiles, afin de limiter les incohérences.
Par exemple, réserver Safari pour les systèmes MacOS, ne pas utiliser IE pour les distributions Linux ...



Dans le fichier .txt, il est possible de spécifier des contraintes en utilisant les expressions IF, THEN, ELSE et les opérateurs logiques NOT, AND, OR.
[Voir la documentation](#), chapitre « CONSTRAINTS ».

Techniques	Contexte	Objectif	Exemple
Partitions d'équivalence	Les résultats attendus varient selon des sets de valeurs.	Réaliser un <i>unique test pour toutes les valeurs du même set</i> (ou partition)	Le taux d'intérêt est de 1,8% si le solde du compte est compris entre 2000 et 3000 €.
Tests aux limites	Les résultats attendus varient d'une partition à l'autre	Réaliser <i>un test par bornes</i> (ou limites) de la partition	Le taux de réduction est appliqué à compter du 25 ^{ème} anniversaire.
Tables de décision	Les résultats attendus varient selon plusieurs partitions s'impactant entre elles	Réaliser <i>un test au croisement de chaque partition</i> .	Pendant les soldes, les clients ont des réductions sur les articles. Pour les clients possédant une carte, une réduction de 5% est toujours appliquée.
All singles	Les résultats attendus varient selon plusieurs variables à valeurs multiples	Réaliser un test contenant <i>au moins une fois</i> chaque valeur de chaque variable	Deux profils (Administrateur, Gestionnaire), trois droits OS (créer, modifier, supprimer) et deux entités (pièces jointes, commentaires) sont utilisés
All pairs		Réaliser <i>un test par paire de variables</i>	

Les techniques de conception des tests vues auparavant s'intéressent surtout aux tests **dynamiques**.

Pour les tests **statiques**, le plus commun est la **revue**.

Une revue peut être organisée pour :

- *Documents de spécification des exigences*
- *Code source*
- *Plan de tests, campagne de tests*
- *Cas de tests*
- *Éléments du product backlog*
- *Chartes de tests (Document encadrant les tests exploratoires)*
- *Documentation du projet*
- *Contrats, modèles*
- ...

→ **Tout produit d'activités qui peut être lu et compris peut faire l'objet d'une revue.**



Les objectifs de la revue dépendent du type de revue, mais globalement peuvent être résumé avec les objectifs suivants :

- Détecter des défauts **dans les premières phases** du cycle de vie du développement logiciel
- **Réduire le temps et les efforts** consacrés à la correction de défauts **à un stade ultérieur** du projet
- Détecter des défauts qui **ne peuvent pas être détectés** par les **tests dynamiques**
- Evaluer la **qualité** des produits d'activités et **construire la confiance** dans ces derniers
- S'assurer que les produits d'activités **décrivent** ou **répondent au besoin réel**
- Créer une **compréhension** et un **langage commun**

→ Prévenir et corriger les défauts au plus tôt, mais aussi créer de l'information.



Les parties prenantes lors des revues peuvent avoir un ou des rôles à jouer lors de celles-ci :

- **Manager** : décide de ce qui doit être revu et fournit les ressources, telles que le personnel et le temps nécessaires à la réalisation de la revue.
- **Auteur** : crée et corrige le produit d'activités en cours de revue.
- **Modérateur** (également appelé **facilitateur**) : veille au déroulement efficace des réunions de revue, y compris la médiation, la gestion du temps et un environnement de revue sûr dans lequel chacun peut s'exprimer librement.
- **Scribe** (également appelé **rapporteur**) : rassemble les anomalies provenant des réviseurs et enregistre les informations relatives à la revue, telles que les décisions et les nouvelles anomalies trouvées au cours de la réunion de revue.
- **Réviseur** : effectue les revues. Un réviseur peut être une personne travaillant sur le projet, un expert en la matière ou toute autre partie prenante.
- **Responsable de la revue** : assume la responsabilité générale de la revue, notamment en décidant qui sera impliqué et en organisant le lieu et la date de la revue.



L'ISTQB distingue 4 types de revues :

- La **revue informelle** : elle ne suit pas un processus défini et n'exige pas de résultat formel documenté.
- Son objectif est principalement la **détection d'anomalies**.
- Exemples :
 - *Revue par les pairs : un membre de l'équipe, par exemple un autre testeur de l'équipe, relit les exigences ou cas de test rédigés par un autre membre de l'équipe*
 - *Programmation par paires : deux développeurs sur un plan de travail*
 - *Revue de conception, de code par un responsable technique*
 - ...



- La **revue technique** : réalisée par des **réviseurs techniquement qualifiés** et dirigée par un **modérateur**.
- Ses objectifs sont :
 - Détecter des anomalies
 - Evaluer la qualité et renforcer la confiance dans le produit d'activité
 - Prendre des décisions concernant un problème technique
 - Parvenir à un consensus
 - Générer de nouvelles idées
 - Motiver les participants et leur permettre de s'améliorer



- La **relecture technique** : réunion menée par l'auteur du produit d'activité révisé. Une revue individuelle avant la relecture technique est possible, mais pas obligatoire.
- Ses objectifs sont :
 - Détecter des anomalies
 - Evaluer la qualité et renforcer la confiance dans le produit d'activité
 - Former les réviseurs
 - Parvenir à un consensus
 - Générer de nouvelles idées
 - Motiver les participants et leur permettre de s'améliorer



- L'**inspection** : le type de revue **le plus formel**, suit le **processus générique complet**. Des métriques sont collectées et utilisées pour **améliorer** le cycle de vie de développement du logiciel. L'**auteur** ne peut pas jouer le rôle de **réviseur** ou de **scribe**.
- Ses objectifs sont :
 - **Détecter des anomalies**
 - **Evaluer la qualité** et renforcer la **confiance** dans le produit d'activité
 - **Motiver les participants** et leur permettre de **s'améliorer**



Le processus de revue passe par les étapes suivantes :

- **Planification** : définition du **périmètre de la revue** :
 - Objectif
 - Produit(s) d'activité(s) à examiner
 - Caractéristiques / qualité à évaluer
 - Domaines à privilégier
 - Critères de sortie
 - Informations complémentaires (standards, effort, délais, ...)



- **Lancement de la revue** : s'assurer que toutes les personnes impliquées sont prêtes.
 - Chaque participant a accès au produit d'activité examiné
 - Chaque participant comprend son rôle et ses responsabilités
 - Chaque participant reçoit tout ce qui est nécessaire à l'exécution de la revue
- **Revue individuelle** : chaque réviseur effectue une revue individuelle pour évaluer la qualité du produit d'activités en cours de revue, en appliquant une ou plusieurs techniques de revue (revue basée sur des checklists, revue basée sur des scénarios).

Les réviseurs enregistrent toutes les anomalies, recommandations et questions qu'ils ont identifiées



- **Communication et analyse** : toutes les anomalies observées pendant les différentes revues individuelles doivent être **analysées** et **discutées**, généralement au sein d'une réunion de revue, pour **valider** (ou non) qu'elles sont bien des **défauts**.

Pour chaque anomalie, il faut décider de leur **statut**, leur **responsable** et des **actions requises**.

Les participants décident également du niveau de qualité du produit d'activités revu. Une **revue de suivi** peut être nécessaire pour mener à bien des actions de suivi nécessaires.

- **Correction et rapport** : pour chaque défaut, un **rapport de défaut** doit être créé, afin que les **actions correctives** puissent faire l'objet d'un suivi.

Lorsque les **critères de sortie** sont atteints, le produit d'activités peut être **accepté**.

Les résultats de la revue font l'objet d'un **rapport**.



Définition tests exploratoires :

- Tests utilisés pour en apprendre davantage sur l'objet de test, pour l'explorer plus en profondeur avec des tests ciblés et pour créer des tests pour les domaines non testés.
- Les tests sont simultanément conçus, exécutés et évalués pendant que le testeur découvre l'objet de test.
- Ils sont utiles lorsque les spécifications sont peu nombreuses ou inadéquates, ou lorsqu'il y a une forte pression de temps.



Quelques bonnes pratiques pour assurer la pertinence des tests exploratoires :

- Le testeur utilise une **charte de test** contenant des **objectifs** de test.
- Les tests sont prévus sur un **périmètre** et un **laps de temps délimité**.
- Les tests exploratoires **complètent d'autres techniques de test** plus formelles, ils ne les remplacent pas.
- Les tests sont **plus efficaces si le testeur est expérimenté** : connaissances dans le domaine, esprit d'analyse, curiosité ou créativité développés, ...
- Le testeur peut utiliser des **formulaires de session de test** pour documenter les **étapes suivies** et les **découvertes** faites.
- La session est généralement **suivie d'un débriefing** entre le testeur et les différentes parties prenantes.

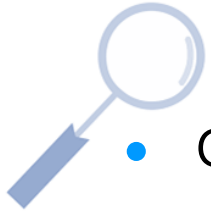
Objectif	Valider la partie paiement de l'application
Durée	5 mai 2025 de 14h00 à 16h00 → 2 heures.
Périmètre	• Se concentrer sur la partie paiement de l'application. • Après une sélection d'items à acheter aller dans le panier et vérifier les prix ainsi que le montant total à payer • Vérifier l'effets des bons de réductions. • Vérifier les surcouts en fonction des types de paiement. • Valider le fonctionnement des différents types de paiement (CB, MasterCard, Paypal, ...) • Références : • Guide Utilisateur • Tableau des prix fournis par le Client • Liste des bons de réductions • Guide sur les différents paiements
Risques	L'accès au serveur des prix du client peut être limité ou lent
Outils	Cartes bancaires de recette disponibles dans le dossier partagé



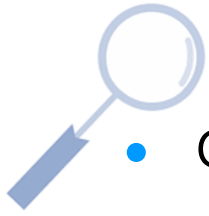
Concevoir les cas de test pour l'application « Newsletter »

1. Lire les spécifications
2. Extraire les exigences
3. Rédiger les Cas de test avec les différents jeux de données

4. Identifier les tests de non-régression



- Qu'est-ce qu'un test de non-régression (appelé aussi « test de régression » ?



- Qu'est-ce qu'un test de non-régression ?



Test de régression :

Tests d'un programme **préalablement testé**, après une **modification**, pour s'assurer que **des défauts n'ont pas été introduits** ou découverts dans des parties non modifiées du logiciel, comme suites des modifications effectuées



Régression :

Dégradation de la qualité d'un composant ou d'un système en raison d'un changement.

NB : Le « changement » peut être lié à la correction d'une anomalie ou à une évolution.

Les **tests de non-régression sont constitués sur la base d'un patrimoine de test existant** avec au moins une version du produit déjà testée et corrigée.

À la différence du patrimoine de test qui cherche à avoir la couverture fonctionnelle la plus exhaustive possible, les tests de non-régression couvrent :

- **les fonctionnalités les plus critiques** (les plus complexes, fonctionnellement et techniquement, les plus utilisées).
- **les fonctionnalités les plus stables.**

A chaque nouvelle mise en production, le patrimoine de non-régression est enrichi de cas de tests **précédemment exécutés avec succès**, couvrant les fonctionnalités éligibles à la MEP.

L'identification des cas de tests éligibles à la non-régression se fait à l'aide d'une **matrice de sélection** prenant en entrée plusieurs critères MOA et MOE auxquels on affecte un coefficient et une « note » allant de 1 à 4. On fait ensuite la somme pour chaque cas de test, et on trie la matrice par note finale décroissante.

Les cas de tests éligibles sont ceux obtenant la **note la plus forte .**

Ceux obtenant une note de 54 et + devront être exécutés à chaque livraison.

Les campagnes de tests de maintenance (v. Syllabus ISTQB chap.2.4 p.45) sont constituées uniquement de TNR !



Cette matrice d'opportunité des TNR ne se limite pas à l'évaluation des cas de test ! Elle peut également être utilisée pour des fonctionnalités ou des macro-fonctionnalités.

Les TNR - MATRICE D'OPPORTUNITÉ DES TNR

Téléchargez le fichier [Matrice d'opportunité TNR_2023.xlsx](#) sur la page de cours Moodle

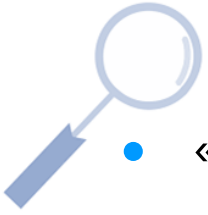
Critères	Définition	Niveaux	Notes	Coeff.
Evaluation de la criticité de la (les) fonction(s)				
Criticité métier de la fonction (à vérifier avec MOA/Métier)	Quelle est la criticité métier de la fonctionnalité ? Quelle est la gravité des impacts si la fonctionnalité n'est pas opérationnelle en production.	Impact critique	4	4
		Impact majeur	3	
		Impact mineur	2	
		Ne sait pas	1	
Fréquence d'utilisation de la fonction (à vérifier avec MOA/Métier)	Quelle est la fréquence d'utilisation de la fonctionnalité ? Est-elle utilisée plusieurs fois par jour, par semaine, par mois, par trimestre, par an ?	Très fréquente	4	4
		Fréquente	3	
		Occasionnelle	2	
		Ne sait pas	1	
Criticité du code associé à la fonction (à vérifier avec MOE/Devs)	Quelle est la complexité du code ?	Code complexe	4	4
		Code moyennement complexe	3	
		Code simple	2	
		Ne sait pas	1	
Evaluation de la stabilité de la (les) fonction(s)				
Stabilité de la fonction (à vérifier avec MOA/Métier + MOE)	Cette fonctionnalité est-elle stable ? (ex : la fonctionnalité est-elle lente, souvent buggée...)	Stable	4	2
		Parfois instable	3	
		Souvent instable	2	
		Ne sait pas	1	
Évolution fonctionnelle/pérennité (à vérifier avec MOA/Métier + MOE)	Cette fonctionnalité sera-t-elle amenée à évoluer dans de prochaines versions ?	Aucune évolution	4	2
		Evolution partielle	3	
		Refonte complète	2	
		Ne sait pas	1	

5. La couverture fonctionnelle

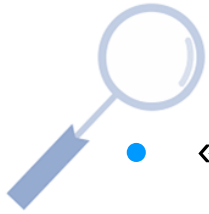


« Exhaustif : qui épuise une matière, qui traite un sujet dans sa totalité. »

Dictionnaire



- « Assurer la couverture fonctionnelle »
- Définition ?



- « Assurer la couverture fonctionnelle »
- Règle d'Or :

**Vérifier que toutes les exigences fonctionnelles
sont couvertes par au moins 1 cas de test.**

- Dans les gestionnaires de tests, **on lie exigences et cas de test**. C'est une relation bijective qui permet la traçabilité et permet de s'assurer de la couverture (« exigences orphelines »).
- Avec SQUASH TM, on peut spécifier cette liaison de deux manières :
 - **Lier un cas de test à une exigence**, dans l'espace Cas de test
 - **Lier une exigence à un cas de test**, dans l'espace Exigences

- La formule pour **calculer** la couverture fonctionnelle est la suivante :

$$\text{Couverture fonctionnelle} = \frac{\text{Nombre d'exigences couvertes par au moins 1 cas de test}}{\text{Nombre total d'exigences}}$$

- **Objectif** : Atteindre les 100%.
- **Réalité** : Certaines exigences peuvent **ne pas être testables** (par ex. environnement, performance, charge, ...). **L'objectif des 100% sera rarement atteint.**

Objectif : 100% des exigences couvertes

- **Cas 1** : un cas de test couvre une exigence
- Exemples :

■ Exigence 1 → ■ Test 1

■ Exigence 2 → ■ Test 2

Exigence

L'application doit permettre une connexion avec un login et un mot de passe valide

Cas de test 1

Étant donné que je suis sur la page de connexion
Quand je rentre un login dans le champ « login »
Et le mot de passe correspondant dans le champ « Mot de passe »
Et que je clique sur le bouton [VALIDER]
Alors j'arrive sur la page d'accueil

Exigence

L'application doit afficher un message d'erreur pour une connexion avec un login et un mot de passe invalide

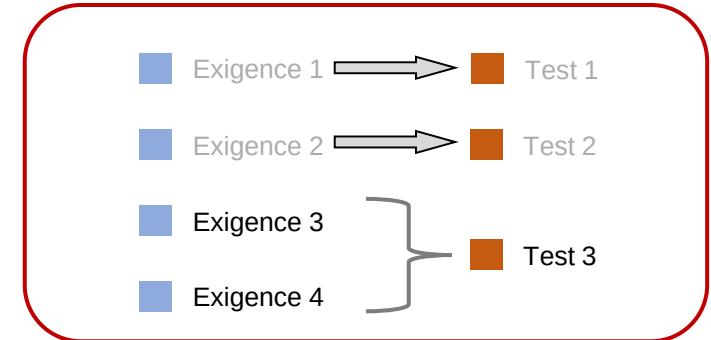
Cas de test 2

Étant donné que je suis sur la page de connexion
Quand je rentre un login dans le champ « login »
Et un mot de passe invalide dans le champ « Mot de passe »
Et que je clique sur le bouton [VALIDER]
Alors je vois un message d'erreur

NB : Ci-contre, les cas de test de haut niveau sont rédigés selon le formalisme Gherkin, couramment utilisé en contexte Agile.

Objectif : 100% des exigences couvertes

- Cas 1 : un cas de test couvre une exigence
- Cas 2 : un cas de test couvre plusieurs exigences
- Exemple :



Exigence 3

L'application doit permettre une connexion avec un login et un mot de passe valide

Exigence 4

L'application doit afficher une page d'accueil conforme à la maquette

Cas de test 3

Etant donné que je suis sur la page de connexion
Quand je rentre un mot de passe dans le champ « Mot de passe »
Et un login dans le champ « login »
Et que je clique sur le bouton [VALIDER]
Alors j'arrive sur la page d'accueil
Et la page d'accueil est conforme à la maquette

Objectif : 100% des exigences couvertes

- Cas 1 : un cas de test couvre **une** exigence
- Cas 2 : un cas de test couvre **plusieurs** exigences
- Cas 3 : une exigence **est couverte par plusieurs** cas de test

Exigence 5

Lorsqu'un utilisateur tente de créer un nouveau compte utilisateur, le système doit vérifier la validité de l'adresse e-mail fournie et s'assurer qu'elle n'est pas déjà utilisée par un autre compte.

Cas de test 5 (nominal)

Création de compte avec une adresse e-mail valide et non utilisée

Étant donné que je suis sur la page d'inscription
Quand je saisis mon nom d'utilisateur, mon mot de passe et mon adresse e-mail
Et que je clique sur le bouton "S'inscrire"
Alors un message de confirmation de création de compte est affiché
Et je suis redirigé vers la page de connexion

Cas de test 6 (alternatif 1)

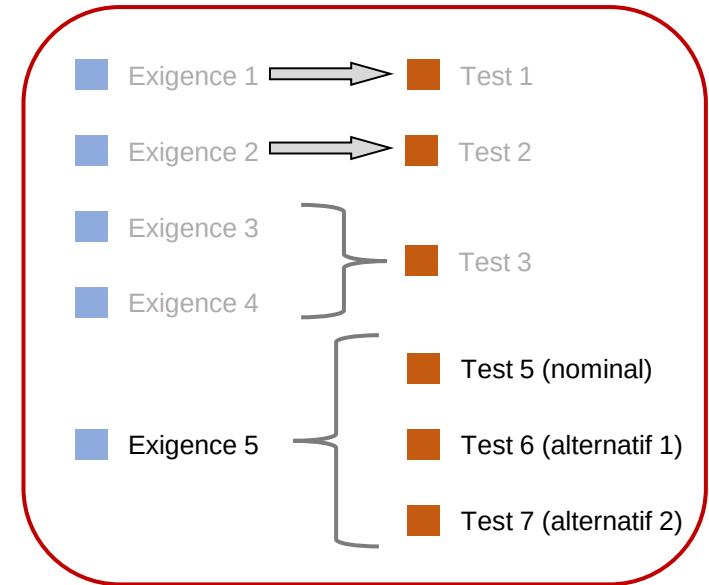
Tentative de création de compte avec une adresse e-mail invalide

Étant donné que je suis sur la page d'inscription
Quand je saisis mon nom d'utilisateur, mon mot de passe et une adresse e-mail invalide
Et que je clique sur le bouton "S'inscrire"
Alors un message d'erreur indiquant que l'adresse e-mail est invalide est affiché
Et je ne suis pas redirigé vers la page de connexion

Cas de test 7 (alternatif 2)

Tentative de création de compte avec une adresse e-mail déjà utilisée

Étant donné qu'il existe un compte utilisateur avec l'adresse e-mail "utilisateur@example.com"
Et que je suis sur la page d'inscription
Quand je saisis mon nom d'utilisateur, mon mot de passe et l'adresse e-mail "utilisateur@example.com"
Et que je clique sur le bouton "S'inscrire"
Alors un message d'erreur indiquant que l'adresse e-mail est déjà utilisée est affiché
Et je ne suis pas redirigé vers la page de connexion



Objectif : 100% des exigences **testables** couvertes

- Cas 1 : un cas de test couvre une exigence
- Cas 2 : un cas de test couvre plusieurs exigences
- Cas 3 : une exigence est couverte par plusieurs cas de test
- Cas 4 : une exigence **non testable** n'est couverte par aucun test

Exigence 6

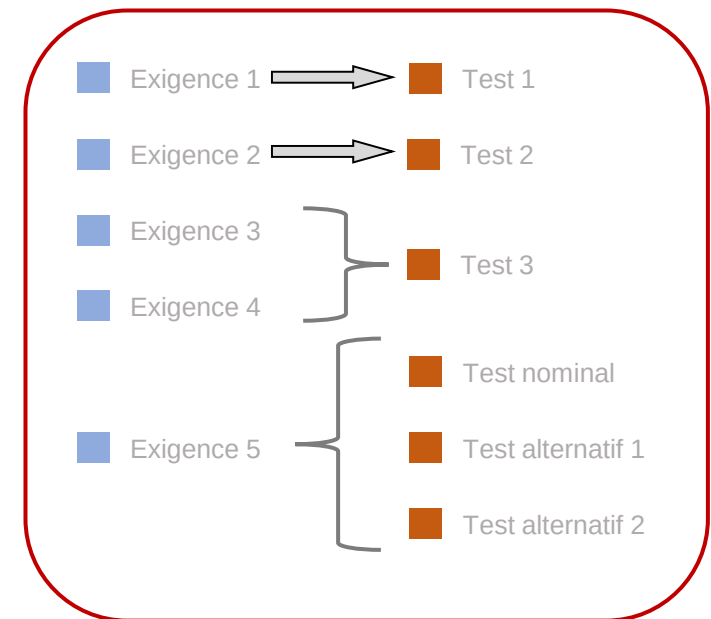
Lorsqu'un utilisateur clique sur le bouton « Sauvegarder », une copie du dossier doit être enregistrée dans le Système d'Information du Service des Ressources Humaines (SI RH)



PROBLÈME : L'environnement de test n'est pas connecté au SI RH, et ne peut pas l'être !

SOLUTION : L'exigence est **non testable**. On ne crée pas de cas de test.

NB : Elle deviendrait testable en environnement de Pré-Production (PréProd), pour lequel on créerait alors les cas de test.



■ Exigence 6 : non testable

- **Erreurs à éviter :**
 - **Si on n'arrive pas à créer un test** pour vérifier une exigence, c'est que **l'exigence est mal décrite**.
 - **Si un cas de test couvre plusieurs exigences**, veiller à leur **criticité** (au cas où le CT soit KO, bloqué, les exigences suivantes pourraient ne pas être vérifiées...)
 - Exigences **critiques** : à traiter **séparément** !
 - Si 1 exigence semble nécessiter de nombreux tests pour être intégralement vérifiée :
 - L'exigence est mal décrite et doit être divisée en plusieurs exigences plus fines.
 - Sinon, optimiser l'effort de test en rédigeant le nombre minimal de cas de test qui mutualisent les manipulations.

- Deux stratégies :
 - **Stratégie productive** : mutualiser les exigences dans les tests
 - Dans 1 seul cas de test, on va vérifier la création, la modification et la suppression d'un compte client.
 - **Stratégie détaillée** (meilleure qualité) :
 - 1 cas de test pour chaque opération.
- La **couverture fonctionnelle** fait partie de la stratégie de recette et doit être définie en fonction des trois critères :
 - Qualité / Coûts / Délais



Exemple d'une mission pour le compte d'un Rectorat

Vous êtes envoyé en mission au Rectorat d'Académie (ministère de l'Éducation nationale) pour tester une plateforme d'inscription à des aides boursières.

- Les bourses sont accordées sur **différents critères sociaux** : situation, revenu, filière, etc.
- Chaque étudiant devra donc renseigner ses informations sur la plateforme.
- La quantité d'informations à renseigner nécessite **plusieurs formulaires** et donc **plusieurs pages** à tester.
- La plateforme doit déduire des informations selon les déclarations et ainsi proposer certaines pages complémentaires ou au contraire en exclure d'autres.



Exemple de mission pour le compte d'un rectorat

Il existe donc différentes
pages/formulaires à tester



Identification



Revenus



Situation familiale



Filière



Informations
complémentaires



Domiciliation

Et différents profils qui obtiennent
des résultats différents



Etudiant 1



Etudiant 2



Etudiant 3

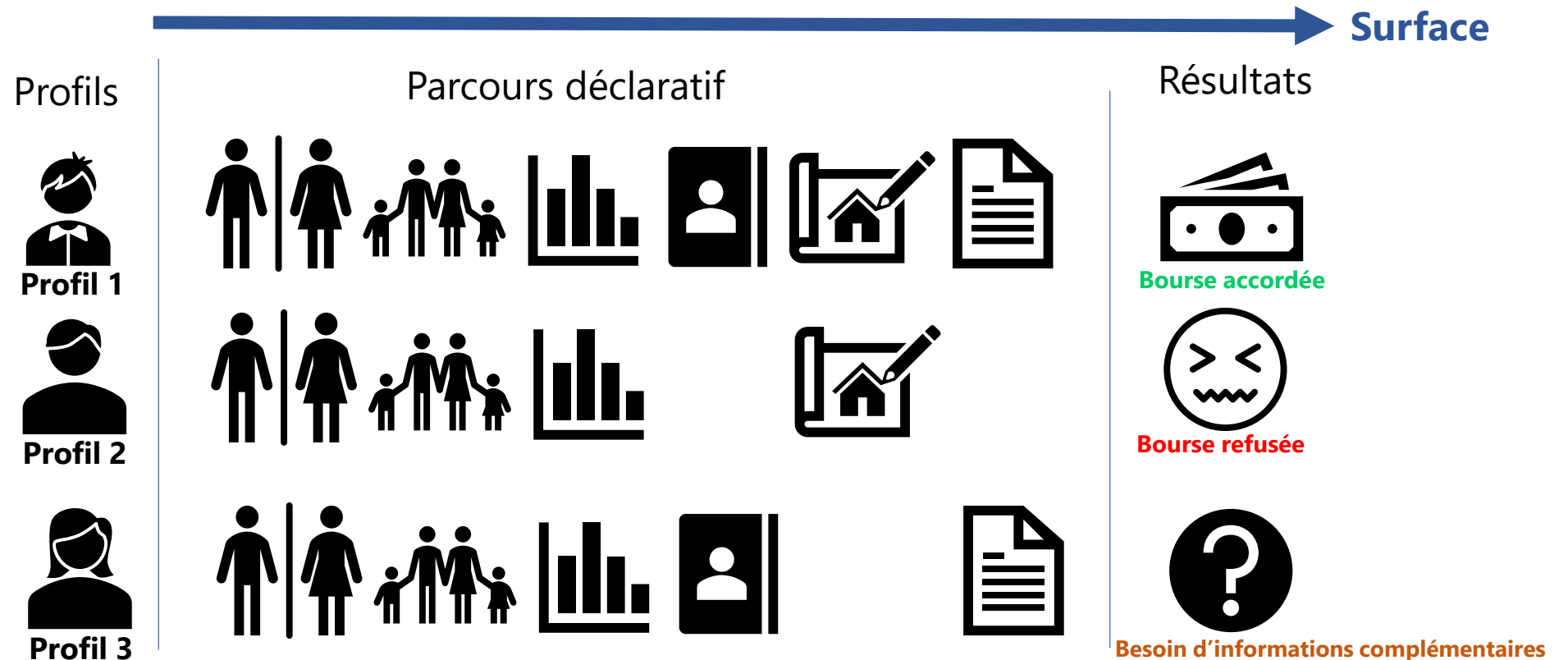
Comment procède-t-on ?



Exemple de mission pour le compte d'un rectorat

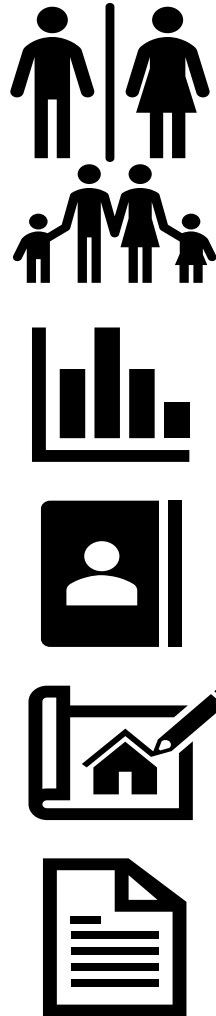
On va identifier les différents profils et anticiper le parcours déclaratif pour chacun d'eux :
on teste ainsi la surface.

On va porter l'effort de tests sur l'enchaînement conforme des pages du formulaire puis sur les résultats en fin de parcours déclaratif





Exemple mission rectorat



Profondeur

On va tester le contenu de chaque page du formulaire indépendamment des réponses du déclarant, il s'agit là de **tester en profondeur** :

- Respect des règles de gestion de chaque champ (casse, nombre de caractères, format, etc.)
- Bon fonctionnement des listes déroulantes, des cases à cocher
- Respect de l'affichage (IHM)
- Ergonomie de la page (tests non-fonctionnels selon ISTQB)



Exemple mission rectorat

Cette méthode permet une couverture fonctionnelle **verticale** (pages /profondeur) et **horizontale** (parcours/surface) et ainsi garantit une réduction des risques.

